# Задание

# Управление формами в React (максимум — 12 баллов)

## Цель

Закрепить навыки работы с формами в React: от базового управления состоянием до динамических полей и схем валидации. Научиться выбирать подходящий инструмент (Formik, React Hook Form или `useActionState`) под конкретную задачу и реализовывать формы, устойчивые к ошибкам.

## Основная часть (до 10 баллов)

## 1. Создание формы регистрации пользователя (4 балла)

### Задание

Создайте страницу с формой регистрации пользователя, включающей поля:

- Имя пользователя (username).
- Email.
- Пароль.
- Подтверждение пароля.

### Что нужно сделать

- Реализовать форму с использованием React Hook Form или Formik.
- Добавить базовую валидацию (можно временно вручную или сразу через схему). Если сразу используете схему — пункты шага 2 считаются выполненными:
    - Все поля обязательны.
    - Email должен содержать `@`.
    - Пароль — минимум 6 символов.
    - Подтверждение пароля должно совпадать с паролем.

### Критерии

- Использован один из рекомендованных подходов (Formik или RHF) — 1 балл.
- Все поля формы обязательны для заполнения — 1 балл.
- Реализована корректная валидация всех полей — 1 балл.

- Ошибки отображаются рядом с соответствующими полями — 1 балл

## 2. Подключение yup или zod (2 балла)

**Задание**

Перенесите валидацию из «ручной» формы в схему. Используйте **yup или zod** по выбору. Если вы сразу использовали схему в Шаге 1, дайте короткий комментарий или ссылку — этот шаг считается выполненным с yup (если используется Formik) или zod (если используется RHF).

**Уточнения**

**Formik**

- Вариант A: `yup.object({...})` через `validationSchema`.
- Вариант B: `zod` — через `validate` (внутри вызывать `schema.safeParse`) или адаптер.

**React Hook Form**

- Используйте любой резолвер: `zodResolver(schema)` или `yupResolver(schema)` из `@hookform/resolvers`.
-

**Критерии**

- Использована схема валидации вместо ручной логики — 1 балл.
- Сообщения об ошибках отображаются корректно рядом с соответствующими полями — 1 балл.

## 3. Динамическое поле: социальные ссылки (4 балла)

**Задание**

Добавьте к форме блок социальных ссылок — список полей, где пользователь может указать свои профили (например, GitHub, GitVerse или соцсети). По умолчанию отображается одно поле, пользователь может добавить или удалить любое количество ссылок.

**Что нужно сделать**

- Использовать `useFieldArray` (в случае RHF) или `FieldArray` (Formik).
- Добавить кнопки «Добавить ссылку» и «Удалить».

**Критерии**

- Использованы `useFieldArray` (в случае RHF) или `FieldArray` (Formik) — 1 балл.
- Форма корректно обрабатывает массив ссылок — 1 балл.
- Добавление и удаление работает без ошибок — 1 балл.
- Каждое поле валидируется как URL — 1 балл:
  - Для Yup: `.url('Некорректный URL')`.
  - Для Zod: `z.string().url('Некорректный URL')`.

## Дополнительное задание (2 бонусных балла)

**Задание**

Реализуйте мини-форму в стиле wizard (2 шага), используя новый встроенный API из React 19 — `useActionState`.

Пример сценария:

1. **Шаг 1** — Ввод email.
2. **Шаг 2** — Подтверждение подписки.

Отображайте UI-состояния:

- `pending` — при отправке;
- `error` — если email не указан или невалидный;
- `success` — сообщение после успешной отправки.

**Критерии**

- Реализована форма с использованием `useActionState` — 1 балл.
- Сообщения об ошибках или успешной отправке отображаются в UI — 1 балл.

## Результат

В результате выполнения задания вы:

- Научитесь реализовывать сложные формы с динамическими блоками.
- Освоите работу с yup и zod для декларативной валидации.

- Получите опыт выбора подходящего инструмента под задачу.